PRÁCTICA GUIADA · WPF & .NET 8

Desarrollo del Directorio Corporativo (CorporateDir WPF)

Arquitectura MVVM profesional con CommunityToolkit.Mvvm e Inyección de Dependencias

1. Objetivos de la Actividad

Arquitectura Limpia

Implementar el patrón **MVVM** desacoplando por completo la lógica de presentación de las vistas C#.

Inyección de Dependencias

Configurar el `DataContext` desde el C# para permitir servicios inyectados por constructor.

Source Generators

Reducir el boilerplate con `[ObservableProperty]` y `[RelayCommand]` del Toolkit.

Diseño XAML Avanzado

Construir layouts horizontales con `Grid` y enlazar colecciones mediante `ListView`.

2. Estructura del Proyecto

Crea una solución WPF en .NET 8 denominada `WpfApp1` e instala el paquete NuGet `CommunityToolkit.Mvvm`. Organiza la estructura de carpetas como se indica:

```
WpfApp1/
├── Models/          --> Clases de entidad (Empleado.cs)
├── Services/        --> Servicios e interfaces mockup (EmpleadoService.cs)
├── ViewModels/      --> Lógica de presentación (MainViewModel.cs)
└── Views/           --> Vistas XAML y Code-Behind (MainWindow.xaml)
```

3. Fase 1: Capa de Dominio y Servicio Simulada

- **Paso 1.1 (Modelo):** Crea la clase `Models/Empleado.cs` con las propiedades: `Id` (int), `Nombre` (string), `Departamento` (string), `Salario` (decimal) y `JornadaCompleta` (bool).
- **Paso 1.2 (Interfaz):** Define la interfaz `Services/IEmpleadoService.cs` con el contrato `List<Empleado> ObtenerEmpleadosMockup()`.
- **Paso 1.3 (Servicio Mockup):** Implementa `Services/EmpleadoService.cs` de forma que retorne una lista inicial con al menos 4 registros simulados de prueba.

> **Regla de Oro: Asignación del DataContext**
>
> Está estrictamente prohibido instanciar el ViewModel dentro del XAML mediante

4. Fase 2: ViewModel e Interfaz XAML

- **Paso 2.1 (MainViewModel):** Crea `ViewModels/MainViewModel.cs` heredando de `ObservableObject`. Inyecta `IEmpleadoService` mediante su constructor y carga los datos en un `ObservableCollection<Empleado>`.
- **Paso 2.2 (Comandos y Propiedades):** Genera las propiedades reactivas `NuevoNombre`, `NuevoDepartamento` y `NuevoActivo` con `[ObservableProperty]`. Añade el método `AgregarEmpleado()` decorado con `[RelayCommand]` y validación de ejecución.
- **Paso 2.3 (Vista XAML):** Diseña `Views/MainWindow.xaml` con una estructura vertical de 3 filas:
  1. **Formulario Horizontal:** Dentro de un `Border`, incluye `TextBox`, `ComboBox` de departamentos, `CheckBox` de jornada y el `Button` enlazado a `AgregarEmpleadoCommand`.
  2. **Listado Principal:** Un `ListView` con `GridView` para mostrar los campos del empleado (formateando el salario con `{0:C}`).
  3. **Barra de Estado:** Un `TextBlock` con `Run` vinculado al `EmpleadoSeleccionado.Nombre`.
- **Paso 2.4 (Code-Behind):** Instancia el servicio y el ViewModel dentro de `MainWindow.xaml.cs`:

```
public MainWindow()
{
    InitializeComponent();
    IEmpleadoService servicio = new EmpleadoService();
    DataContext = new MainViewModel(servicio);
}
```

> **Entregable Requerido**
>
> Comprime la carpeta del proyecto omitiendo los directorios `bin` y `obj`. Adjunta una captura de pantalla donde se observe el formulario agregando un registro dinámicamente a la tabla.